DiabAI

Diabetes Prediction System

Healthcare ML Application

Test Case Suite (TCS) Report

Software Quality Assurance (SQA) Documentation

Document ID	TCS-DiabAI-v1.0
Version	1.0-Quality-Audit
Status	**Released**
Testing Type	Black-Box / Manual
Tools Used	Jira, Postman, Chrome DevTools, Python
Jira Epic	DiabAI: SQA Audit (DS-1)
Bugs Documented	DS-25 (Logical Validation), DS-26 (Mismatch)
Total Test Cases	18
Total Stories	5 (DS-2, DS-8, DS-14, DS-17, DS-21)

Prepared by: Syed Ammar Zulfiqar | Classification: Portfolio / Public | March 31, 2026

Contents

1 Introduction **1**
1.1 Story Summary . . . 1
1.2 Bug Summary . . . 1

2 Story DS-2: Input Validation Testing **2**
2.1 TC-01 – Validate Pregnancies Field (0–20 Range) . . . 2
2.2 TC-02 – Validate Glucose Field (0–250 Range) . . . 3
2.3 TC-03 – Validate BMI Field (0–100 Range) . . . 4
2.4 TC-04 – Validate Insulin Field (0–1000 Range) . . . 5
2.5 TC-05 – Validate Age Field (0–100 Range) . . . 6

3 Story DS-8: Prediction Functionality Testing **7**
3.1 TC-06 – Test Prediction with Valid High-Risk Inputs . . . 7
3.2 TC-07 – Test Prediction with Edge / Borderline Cases . . . 8
3.3 TC-08 – Verify Prediction Probability Output Sensitivity . . . 9
3.4 TC-09 – Test SHAP Explanation Generation . . . 10
3.5 TC-10 – Test Permutation Importance Display . . . 11

4 Story DS-14: Performance Testing **12**
4.1 TC-11 – Response Time for Prediction (Latency Test) . . . 12
4.2 TC-12 – UI/UX Responsiveness Under Stress . . . 13

5 Story DS-17: UI/UX Testing **14**
5.1 TC-13 – Test All Input Fields Are Visible and Accessible . . . 14
5.2 TC-14 – Test Model Performance Metrics Dashboard . . . 15
5.3 TC-15 – Production Deployment & CI/CD Pipeline Verification . . . 16

6 Story DS-21: API Testing **17**
6.1 TC-16 – Test /predict Endpoint Security and Reachability . . . 17
6.2 TC-17 – Test Request/Response Format Integrity . . . 18
6.3 TC-18 – Test Error Handling on Backend Failure . . . 19

7 Test Execution Summary **20**

1. Introduction

This document is the Test Case Suite (TCS) for the DiabAI Diabetes Prediction System, a healthcare Machine Learning web application that predicts the likelihood of diabetes based on eight clinical parameters. It maps all 18 sub-tasks from the Jira Epic **DiabAI: SQA Audit (DS-1)** to structured, reproducible test cases across all five User Stories.

1.1 Story Summary

Jira ID	Story Title	Scope
DS-2	**Input Validation Testing**	Boundary value testing of all 8 clinical input fields.
DS-8	**Prediction Functionality Testing**	ML model output, edge cases, SHAP, permutation importance.
DS-14	**Performance Testing**	Response latency, UI responsiveness under stress.
DS-17	**UI/UX Testing**	Field visibility, model metrics dashboard, CI/CD deployment.
DS-21	**API Testing**	Endpoint security, request, response integrity, error handling.

1.2 Bug Summary

Bug ID	Title	Severity	Status
DS-25	Logical Validation Gap: System accepts '0' for Glucose, BMI, and Age	**High**	**Done**
DS-26	Prediction Logic Mismatch: UI displays "Diabetes" for probability $< 50\%$	**High**	**Done**

Severity is assigned using the defect-driven classification model defined in `STP-DiabAI-v1.0`. High priority denotes test cases where a failure produces a clinically incorrect output or exposes a security vulnerability. Low priority denotes stable functional paths, UI/UX checks, and performance benchmarks.

2. Story DS-2: Input Validation Testing

2.1 TC-01 – Validate Pregnancies Field (0–20 Range)

Field Detail	
Test Case ID	TC-01
Jira Sub-Task	DS-3: 2.1 Validate Pregnancies field (0–20 range)
Related Story	DS-2 (Input Validation Testing)
Precondition	Flask/Streamlit server is running on localhost. Application UI is fully loaded in browser.
Test Steps	1. Open the DiabAI web interface. 2. Locate the Pregnancies input stepper. 3. Attempt to increment the value beyond 20 using the stepper button. 4. Attempt to type a value of 21 or higher directly in the field. 5. Observe button state and field value.
Test Data	Pregnancies = 0 (lower bound), 20 (upper bound), 21 (out-of-range), -1 (negative)
Expected Result	Field must reject values > 20. Stepper '+' button disables at 20. Field clamps at boundary.
Actual Result	Field successfully blocked values > 20. Stepper buttons disabled at the limit. Values 21 and -1 were rejected.
Status	**PASS**
Linked Bug	None

2.2 TC-02 – Validate Glucose Field (0–250 Range)

Field Detail	
Test Case ID	TC-02
Jira Sub-Task	DS-4: 2.2 Validate Glucose field (0–250 range)
Related Story	DS-2 (Input Validation Testing)
Precondition	Flask/Streamlit server is running. Application UI fully loaded.
Test Steps	1. Open the DiabAI web interface. 2. Locate the Glucose input field. 3. Enter a value of 0 and observe live prediction. 4. Enter a value of 251 (out of range). 5. Enter a value of -1 and observe field behavior. 6. Enter a valid boundary value of 1 and 250. 7. Observe system response and any validation messages.
Test Data	Glucose = -1 (negative), 0, 1, 148 (typical high-risk), 250, 251.
Expected Result	System should reject Glucose = 0 as medically impossible. Field should reject values > 250. A validation error or prevention must be shown.
Actual Result	System accepted Glucose = 0 without error and produced a prediction. Upper bound validation functioned correctly. Negative value (-1) was also rejected by the field boundary enforcement. Zero-value accepted erroneously. Bug filed.
Status	**BUG – DS-25**
Linked Bug	DS-25 (Logical Validation Gap)

2.3 TC-03 – Validate BMI Field (0–100 Range)

Field Detail	
Test Case ID	TC-03
Jira Sub-Task	DS-5: 2.3 Validate BMI field (0–100)
Related Story	DS-2 (Input Validation Testing)
Precondition	Application running locally. User is on the main prediction form page.
Test Steps	1. Navigate to the BMI input slider/field. 2. Set BMI = 0 and observe prediction behavior. 3. Set BMI = 70 (physiological maximum extreme per BVA). 4. Attempt to enter BMI = 101 (out of range). 5. Enter a value of -1 and observe field behavior. 6. Enter valid BVA boundary values: 0.1 and 100.
Test Data	BMI = -1 (negative), 0, 0.1, 33.6 (typical), 70, 100, 101.
Expected Result	BMI = 0 must be rejected as medically impossible. Values > 100 must be blocked.
Actual Result	BMI = 0 was accepted by the system and passed to the model. Out-of-range upper limit was blocked correctly. Negative value (-1) was rejected. Zero-value gap confirmed for BMI specifically.
Status	**BUG – DS-25**
Linked Bug	DS-25 (Logical Validation Gap)

2.4 TC-04 – Validate Insulin Field (0–1000 Range)

Field Detail	
Test Case ID	TC-04
Jira Sub-Task	DS-6: 2.4 Validate Insulin field (0–1000)
Related Story	DS-2 (Input Validation Testing)
Precondition	Application running. Prediction form accessible.
Test Steps	1. Go to the Insulin input field. 2. Set Insulin = 0 and observe behavior. 3. Set Insulin = 1000 (upper boundary). 4. Attempt to set Insulin = 1001. 5. Record whether stepper clamps or allows overflow.
Test Data	Insulin = 0, 1, 79 (typical), 1000, 1001
Expected Result	Values > 1000 must be blocked. Stepper must disable at upper limit.
Actual Result	Upper bound enforcement at 1000 functioned correctly. Stepper disabled at max. All invalid out-of-range inputs rejected.
Status	**PASS**
Linked Bug	None

2.5 TC-05 – Validate Age Field (0–100 Range)

Field Detail	
Test Case ID	TC-05
Jira Sub-Task	DS-7: 2.5 Validate Age field (0–100)
Related Story	DS-2 (Input Validation Testing)
Precondition	Application running. Prediction form accessible.
Test Steps	1. Navigate to the Age input stepper. 2. Enter Age = 0 and observe model prediction. 3. Enter Age = 1 (minimum realistic value). 4. Enter Age = 100 (upper boundary). 5. Enter Age = -1 and observe field response. 6. Attempt Age = 101 (out of range).
Test Data	Age = -1 (negative), 0, 1, 33 (typical), 100, 101.
Expected Result	Age = 0 must be rejected (no living patient has age 0 in a diabetes screening context). Values > 100 blocked.
Actual Result	Negative value (-1) was correctly rejected. Zero-value (Age = 0) was accepted erroneously. Bug filed.
Status	**BUG – DS-25**
Linked Bug	DS-25 (Logical Validation Gap)

3. Story DS-8: Prediction Functionality Testing

3.1 TC-06 – Test Prediction with Valid High-Risk Inputs

Field	Detail
Test Case ID	TC-06
Jira Sub-Task	DS-9: 8.1 Test prediction with valid inputs
Related Story	DS-8 (Prediction Functionality Testing)
Precondition	Application running. model.pkl is loaded. All input fields are accessible.
Test Steps	1. Enter the clinically significant high-risk profile below. 2. Submit the form / allow live prediction. 3. Observe the probability value and result classification label. 4. Confirm the label reads "Diabetic" or equivalent positive result.
Test Data	Pregnancies=6, Glucose=148, Insulin=79, BMI=33.6, Age=50
Expected Result	Model outputs a positive "Diabetic" classification. Probability score > 50%.
Actual Result	Model correctly classified as Diabetic. Probability output: 68.94%. Core predictive functionality confirmed operational.
Status	**PASS**
Linked Bug	None

3.2 TC-07 – Test Prediction with Edge / Borderline Cases

Field	Detail
Test Case ID	TC-07
Jira Sub-Task	DS-10: 8.2 Test prediction with edge cases
Related Story	DS-8 (Prediction Functionality Testing)
Precondition	Application running. Bug DS-25 validation in place (or noted if not).
Test Steps	1. Enter borderline clinical values (see Test Data below). 2. Observe the prediction probability produced by the model. 3. Cross-check the displayed UI label ("Diabetic" / "Healthy") against the probability. 4. Verify that probability $< 50\%$ is labeled "Healthy" and vice versa.
Test Data	Pregnancies=2, Glucose=125, Insulin=0, BMI=29.5, Age=35
Expected Result	Probability $< 50\%$; UI must display Healthy/ Non-Diabetic result. Classification threshold logic: probability $\geq 50\% \rightarrow$ Diabetic, probability $< 50\% \rightarrow$ Healthy (No Diabetes). This must be consistently enforced across the UI label, result banner, and any exported output.
Actual Result	Model returned probability of 35.09%. However, UI displayed "Diabetes" label despite probability being below 50%. Mismatch confirmed – Bug filed.
Status	**BUG – DS-26**
Linked Bug	DS-26 (Prediction Logic Mismatch)

3.3 TC-08 – Verify Prediction Probability Output Sensitivity

Field	Detail
Test Case ID	TC-08
Jira Sub-Task	DS-11: 8.3 Verify prediction probability output
Related Story	DS-8 (Prediction Functionality Testing)
Precondition	Application running. Prediction outputs visible in the probability gauge.
Test Steps	1. Enter a high-risk profile (from TC-06). 2. Note the probability (e.g., 68.94%). 3. Progressively improve health markers (lower Glucose, lower BMI, lower Age). 4. Re-observe probability after each change. 5. Verify probability decreases as health indicators improve.
Test Data	Profile 1: Glucose=148, BMI=33.6, Age=50 Profile 2: Glucose=90, BMI=22.0, Age=30 Profile 3: Glucose=70, BMI=18.5, Age=21
Expected Result	Probability should decrease monotonically as clinical parameters improve toward healthy norms.
Actual Result	Probability decreased linearly: 68.94% → 31.2% → 12.5%. Model sensitivity to low-risk data confirmed.
Status	**PASS**
Linked Bug	None

3.4 TC-09 – Test SHAP Explanation Generation

Field	Detail
Test Case ID	TC-09
Jira Sub-Task	DS-12: 8.4 Test SHAP explanation generation
Related Story	DS-8 (Prediction Functionality Testing)
Precondition	Application running with SHAP visualization component enabled. High-risk profile from TC-06 entered.
Test Steps	1. Enter the high-risk input values from TC-06 (Glucose=148, BMI=33.6, Age=50). 2. Trigger prediction. 3. Navigate to or observe the SHAP waterfall / force plot. 4. Verify that Glucose and BMI appear as the longest red bars (risk-increasing features). 5. Verify feature names and values are accurately labeled on the chart.
Test Data	Same as TC-06: Pregnancies=6, Glucose=148, Insulin=79, BMI=33.6, Age=50
Expected Result	SHAP plot renders correctly. Glucose and BMI are top contributors pushing risk upward (red bars).
Actual Result	SHAP plot generated successfully. BMI returned the highest positive SHAP value **(+0.1)**, followed by Glucose **(+0.05)**, confirming both as the dominant risk-increasing features for this input profile. Feature labels, input values, and directional arrows were accurately rendered on the waterfall plot.
Status	**PASS**
Linked Bug	None

3.5 TC-10 – Test Permutation Importance Display

Field	Detail
Test Case ID	TC-10
Jira Sub-Task	DS-13: 8.5 Test permutation importance display
Related Story	DS-8 (Prediction Functionality Testing)
Precondition	Application running. Permutation importance chart visible in the UI.
Test Steps	1. Trigger a prediction using the high-risk profile (TC-06 values). 2. Navigate to the Permutation Importance chart/section. 3. Verify the bar chart loads without error. 4. Confirm Glucose and BMI are ranked in the top 2 features by importance score. 5. Verify the chart is static (global model-level, not per-prediction).
Test Data	Same as TC-06: Glucose=148, BMI=33.6, Age=50 (to trigger a full prediction cycle)
Expected Result	Permutation importance chart renders. Glucose and BMI are highest-ranked features, consistent with model training priorities.
Actual Result	Chart rendered correctly. Glucose ranked #1, BMI ranked #2 in global feature importance. Model training priorities confirmed accurate.
Status	**PASS**
Linked Bug	None

4. Story DS-14: Performance Testing

4.1 TC-11 – Response Time for Prediction (Latency Test)

Field	Detail
Test Case ID	TC-11
Jira Sub-Task	DS-15: 14.1 Response time for prediction
Related Story	DS-14 (Performance Testing)
Precondition	Application running on localhost. Chrome DevTools (Network tab) open and recording.
Test Steps	1. Open Chrome DevTools → Network tab → enable "Preserve log". 2. Enter any valid input set and trigger a prediction. 3. Locate the XHR/Fetch prediction request in the network log. 4. Record the Round Trip Time (RTT) shown in the Timing tab. 5. Rapidly update Glucose slider 10 times in succession to simulate stress. 6. Record RTT for each request and observe for degradation.
Test Data	Glucose varied: 80, 100, 120, 140, 160, 180, 200, 220, 240, 148 (stress cycle)
Expected Result	Each prediction request should complete between **260–430ms** (observed baseline). This falls well within the 1000ms interactive response threshold defined by **Nielsen's usability heuristics** for web applications, and is acceptable for a single-user local ML inference workload. No memory leaks or process hangs under rapid sequential requests.
Actual Result	Average **RTT** measured at approximately 260–430ms per request, recorded via Chrome DevTools Network tab (Timing panel). No degradation observed under 10-request stress cycle. Backend remained stable throughout.
Status	**PASS**
Linked Bug	None

4.2 TC-12 – UI/UX Responsiveness Under Stress

Field Detail	
Test Case ID	TC-12
Jira Sub-Task	DS-16: 14.2 UI/UX responsiveness
Related Story	DS-14 (Performance Testing)
Precondition	Application running. Chrome DevTools Performance tab available for profiling.
Test Steps	1. Open Chrome DevTools → Performance tab → start recording. 2. Rapidly manipulate Glucose and Insulin sliders for 10+ seconds. 3. Stop recording and analyze the flame chart. 4. Look for "Long Tasks" (> 50ms) on the main thread. 5. Visually observe: does the probability circle and result banner update smoothly?
Test Data	Glucose: swept 70–240 repeatedly. Insulin: swept 0–800 repeatedly over 10 seconds.
Expected Result	No long tasks (> 50ms) that freeze the UI. Probability gauge updates fluidly. No visual jank or layout shifts.
Actual Result	Performance profiling showed no blocking Long Tasks. UI remained interactive and smooth. Probability circle updated in real-time without stutter.
Status	**PASS**
Linked Bug	None

5. Story DS-17: UI/UX Testing

5.1 TC-13 – Test All Input Fields Are Visible and Accessible

Field Detail	
Test Case ID	TC-13
Jira Sub-Task	DS-18: 17.1 Test all input fields are visible
Related Story	DS-17 (UI/UX Testing)
Precondition	Application running. Browser viewport at standard desktop resolution (1920×1080) and reduced widths.
Test Steps	1. Open DiabAI in browser at 1920×1080. 2. Cross-reference UI fields against the 5 required model features: Pregnancies, Glucose, Insulin, BMI, Age. 3. Resize viewport to 1280px, then 768px (tablet) width. 4. Verify sidebar and input panel do not overlap the "DiabAI" header. 5. Confirm SHAP/probability visualizations do not obscure interactive input sliders.
Test Data	Screen widths tested: 1920px, 1280px, 1024px, 768px
Expected Result	All 5 input fields visible, correctly labeled, and interactable at all viewport widths tested. No z-index or overlap issues.
Actual Result	All 5 fields visible and correctly labeled across all tested widths. Responsive sidebar behavior confirmed. No element overlap detected with SHAP or probability gauge.
Status	**PASS**
Linked Bug	None

5.2 TC-14 – Test Model Performance Metrics Dashboard

Field	Detail
Test Case ID	TC-14
Jira Sub-Task	DS-19: 17.2 Test model performance metrics dashboard
Related Story	DS-17 (UI/UX Testing)
Precondition	Application running. Model performance section of the UI is accessible. Pre-computed model evaluation logs available for cross-reference.
Test Steps	1. Navigate to the Model Performance dashboard section. 2. Visually inspect all 5 circular gauges: Accuracy, F1-Score, Recall, Precision, ROC-AUC. 3. Verify percentage formatting and text alignment within each gauge. 4. Vary input sliders (Glucose, BMI) and confirm these global metrics do NOT change (they are static). 5. Cross-reference Accuracy value displayed against pre-computed log: expected 81.9%.
Test Data	Expected static values: Accuracy=81.9%, F1=0.76, Recall=0.74, Precision=0.79, ROC-AUC=0.88
Expected Result	All 5 gauges render correctly with proper formatting. Values remain static as inputs change. Accuracy matches 81.9% baseline.
Actual Result	All gauges rendered without error. Values remained static during input changes. Accuracy displayed as 81.9%, consistent with model evaluation logs.
Status	**PASS**
Linked Bug	None

5.3 TC-15 – Production Deployment & CI/CD Pipeline Verification

Field	Detail
Test Case ID	TC-15
Jira Sub-Task	DS-20: 17.3 Production Deployment & CI/CD Pipeline verification
Related Story	DS-17 (UI/UX Testing)
Precondition	GitHub repository with CI/CD pipeline configured for auto-deploy on push to main. Railway.app account with trial credits.
Test Steps	1. Push a code commit to the main branch of the GitHub repository. 2. Monitor the GitHub Actions / Railway deployment log. 3. Verify build completes with 0 errors and 100% success rate. 4. Access the live production URL: `web-production-efc90.up.railway.app`. 5. Confirm the application loads and prediction functionality works in the cloud environment. 6. Document current offline status due to trial credit expiry.
Test Data	Production URL: `https://web-production-efc90.up.railway.app/` Deployment verified on **7 February 2026**.
Expected Result	CI/CD pipeline completes successfully. Application accessible and functional at the production URL.
Actual Result	Build and deployment succeeded with 100% success rate (confirmed via GitHub logs). Application was live and functional on 7 February 2026. Currently offline due to Railway trial credit exhaustion, not a code defect. Re-verification required upon credit renewal before this test case can be closed as unconditional PASS.
Status	**PASS (Conditional)**
Linked Bug	None

6. Story DS-21: API Testing

6.1 TC-16 – Test /predict Endpoint Security and Reachability

Field	Detail
Test Case ID	TC-16
Jira Sub-Task	DS-22: 21.1 Test /predict endpoint
Related Story	DS-21 (API Testing)
Precondition	Application running locally. Postman installed. Chrome DevTools Network tab available.
Test Steps	1. Open Chrome DevTools Network tab and trigger a prediction from the UI. 2. Identify the internal prediction XHR/Fetch request. 3. Open Postman and attempt a direct POST request to the /predict route on localhost. 4. Observe the server response to the external Postman request. 5. Verify CORS / XSRF headers in the server response. 6. Send a malformed POST request via Postman with a string value in a numeric field e.g., `"glucose": "abc"`. 7. Send a request with a missing required field e.g., `omit "bmi" entirely from the body`. 8. Send a null body request Body e.g., `{}`. 9. Observe and record the server response for each malformed case.
Test Data	Postman POST to: `http://localhost:8501/` (or equivalent Streamlit endpoint) Headers: `Content-Type: application/json` Body: `{"glucose": 148, "bmi": 33.6}`
Expected Result	Internal requests succeed. External Postman request is blocked by CORS/XSRF protection. Malformed payloads (wrong types, missing fields, null body) must be rejected with an appropriate error response. No stack trace or internal path must be exposed.
Actual Result	Malformed payload tests confirmed: string input in numeric field, missing field, and null body were all rejected by the backend. External Postman request was rejected by CORS protection. Server returned error responses without exposing stack traces or internal file paths.
Status	**PASS**
Linked Bug	None

6.2 TC-17 – Test Request/Response Format Integrity

Field	Detail
Test Case ID	TC-17
Jira Sub-Task	DS-23: 21.2 Test request/response format
Related Story	DS-21 (API Testing)
Precondition	Application running. Chrome DevTools Network tab open.
Test Steps	1. Enter valid clinical data across all 8 fields. 2. Trigger a prediction. 3. Inspect the outgoing data payload in DevTools Network → Request Payload. 4. Verify all 8 features are bundled correctly as numeric types (Float/Int). 5. Inspect the server response payload and cross-reference the probability value against the UI probability gauge. 6. Check for any TypeErrors logged in the console.
Test Data	All 5 fields: Pregnancies=2, Glucose=100.0, Insulin=100.0, BMI=37.0, Age=25
Expected Result	All 5 features transmitted as correct numeric types. Response probability matches UI display. No TypeErrors in console.
Actual Result	Data serialization verified – all features transmitted as correct types. Response probability matched UI gauge exactly. No TypeErrors logged.
Status	**PASS**
Linked Bug	None

6.3 TC-18 – Test Error Handling on Backend Failure

Field	Detail
Test Case ID	TC-18
Jira Sub-Task	DS-24: 21.3 Test error handling
Related Story	DS-21 (API Testing)
Precondition	Application running with the frontend session active in browser. Chrome DevTools Network tab open and preserving log.
Test Steps	1. With the frontend active, manually terminate the local Streamlit server process (Ctrl+C or kill process). 2. Attempt to interact with the UI – move a slider or trigger a prediction. 3. Monitor the Network tab for failed requests (e.g., health, host.config). 4. Observe the UI feedback: does the application display a user-friendly error message? 5. Verify no raw Python traceback or file system paths are exposed to the user.
Test Data	Simulated server crash: Streamlit process killed while session active
Expected Result	Network requests fail gracefully. UI displays a user-friendly "Connection lost" or equivalent message. No stack traces or internal paths exposed.
Actual Result	Network tab showed FAILED: Connection error on health and host.config heartbeat calls. Streamlit's built-in "Connecting…" overlay appeared. No stack traces or internal paths were exposed to the user.
Status	**PASS**
Linked Bug	None

7. Test Execution Summary

TC ID	Title	Story	Priority	Status / Bug
TC-01	Validate Pregnancies field (0–20)	DS-2	Low	PASS –
TC-02	Validate Glucose field (0–250)	DS-2	**High**	BUG – DS-25
TC-03	Validate BMI field (0–100)	DS-2	**High**	BUG – DS-25
TC-04	Validate Insulin field (0–1000)	DS-2	Low	PASS –
TC-05	Validate Age field (0–100)	DS-2	**High**	BUG – DS-25
TC-06	Prediction with valid high-risk inputs	DS-8	Low	PASS –
TC-07	Prediction with borderline edge cases	DS-8	**High**	BUG – DS-26
TC-08	Prediction probability output sensitivity	DS-8	Low	PASS –
TC-09	SHAP explanation generation	DS-8	Low	PASS –
TC-10	Permutation importance display	DS-8	Low	PASS –
TC-11	Response time for prediction	DS-14	Low	PASS –
TC-12	UI/UX responsiveness under stress	DS-14	Low	PASS –
TC-13	All input fields visible and accessible	DS-17	Low	PASS –
TC-14	Model performance metrics dashboard	DS-17	Low	PASS –
TC-15	Production deployment & CI/CD	DS-17	Low	PASS (Cond.)
TC-16	/predict endpoint security	DS-21	Low	PASS –
TC-17	Request/response format integrity	DS-21	Low	PASS –
TC-18	Error handling on backend failure	DS-21	Low	PASS –
Total: 18 Test Cases				**14 PASS 4 BUG**

Audit Sign-off: This Software Test Plan was executed with all criteria met. Critical bugs **DS-25** and **DS-26** were identified, prioritised, and documented with attachments, The `v1.0-Quality-Audit` release has been formally tagged in Jira.